

ConfluenceGuessr Handover Guide

This is a significantly more comprehensive guide on how to go about playing ConfluenceGuessr as well as even doing further development on it. The [README](#) page on the ConfluenceGuessr_Handover git repo will be enough if you just want to simply play ConfluenceGuessr.

1. Accounts, keys and third-party services

1.1 What you need before you start

ConfluenceGuessr is handed over as source code. Before it can be deployed, the following need to be created:

Item	Purpose	Where to get it from
Atlassian account with Forge CLI	Deploy and Install the app	<pre>npm install -g @forge/cli</pre> then <pre>forge login</pre>
Atlassian site (Confluence)	Where the app is installed	Existing Confluence Cloud site, or a free dev site
Forge app ID	Uniquely identifies copy of app	<pre>forge register</pre>
Groq API Key	Primary AI provider for question generation	Groq Console (console.groq.com) : Free tier
Google Gemini API Key	Fallback AI provider	Google AI Studio (aistudio.google.com) : Free tier
Node.js	Matches the app runtime in <code>manifest.yml</code>	NodeJS (nodejs.org)

1.2 Registering your own app ID

The `manifest.yml` file we hand over has our app ID removed. Before you deploy, first run:

```
forge register
```

This creates a new app under your Atlassian account and adds the new ID into `manifest.yml`. You can do this only do this once.

1.3 Setting up the API keys

The two AI provider API keys are stored as encrypted Forge environment variables and not in our source code. Set them once per environment by running these commands:

```
forge variables set --encrypt GROQ_API_KEY
forge variables set --encrypt GEMINI_API_KEY
```

You will be prompted to paste the values for both. Repeat for each environment you deploy to as these variables are not shared between environments.

1.4 The app still runs without AI keys

If either of the API keys is missing, the provider is skipped rather than throwing an error. If both are missing, the app falls back to generating questions locally from page text. The game remains fully playable and still produces the exact number of questions a user asks for. The questions are just chosen mechanically instead of using AI.

This is intentional design so that the product never breaks. The trade-off of this is that a missing key is easy to miss, so checking variables before deployment is a must.

1.5 Free tier limits

Both providers are on free tiers:

Provider	Model	Free tier limits
Groq	openai/gpt-oss-20b	8,000 tokens/minute, 1,000 requests/day
Google Gemini	gemini-3.6-flash	20 requests/day

A single 20-question game uses up roughly 2 AI requests. Gemini’s daily limit is not published in Google’s official documentation and we identified it from the API’s own quota error during testing.

Recommendation: Move Groq to a paid tier, or replace the provider entirely. The providers are built to be swappable which means adding one means just writing a single call function and adding its domain to the egress list in `manifest.yml`.

Note: Rate limits are enforced per organisation or project and not per key. Generating a new key on the same account does not reset an exhausted quota.

1.6 What leaves your Confluence site

The app sends the plain text of pages the player selects to Groq or Gemini, capped at 900 characters per page. It does not send any sensitive information such as page titles, usernames, account IDs, images or attachments.

All Confluence content is fetched as the logged-in user, so the app cannot read pages that the user does not have access to. Image questions never touch an AI provider at all and are built entirely inside Confluence.

Forge restricts outbound network access to domains declared in `manifest.yml`. Current state of the file:

```
permissions:
  external:
    fetch:
      backend:
        - address: 'api.groq.com'
        - address: 'generativelanguage.googleapis.com'
```

The Confluence admin sees this list when they approve the install of the app.

1.7 Confluence permissions requested

Scope	Why it is needed
<code>read:page:confluence</code>	Read page content to generate question
<code>read:space:confluence</code>	List spaces the player has access to
<code>read:attachment:confluence</code>	Fetch images for image questions
<code>read:confluence-space.summary</code>	Space name for selection screen
<code>read:confluence-content.summary</code>	Page titles for answer options
<code>storage:app</code>	Store scores and leaderboard data

The app requests read-only access to Confluence content and it never creates, edits or delete pages.

2. Deploying the app

2.1 Order of operations

The app contains a backend and a frontend that are installed separately, And the frontend must be built before the first deploy. Running the steps are of order will result in a failed deployment.

1. Install backend dependencies
2. Install frontend dependencies
3. Build the frontend
4. Deploy
5. Install on Confluence site

2.2 Installing dependencies

There are a total of two `package.json` files and both of them need installing. From the project root, run the following commands:

```
npm install
cd static
npm install
cd ..
```

Note: If `npm install` fails with a peer dependency error, run it with `--legacy-peer-deps`. This is caused by a version conflict between the Forge storage library and the frontend build. It does not affect the running app though.

2.3 Building frontend

The `manifest.yml` points the app's user interface at `static/build`

```
resources:
- key: main
  path: static/build
```

That folder is a build artifact and is excluded from the repository and does not arrive with the source code. It must be generated before the first deploy by running these commands:

```
cd static
npm run build
cd ..
```

Note: If this step is skipped for whatever reason, `forge deploy` (or `forge tunnel`) fails with an error stating that `index.html` is missing from `static/build`. The fix is to run the build, not to change the manifest.

2.4 Deploying and installing

Deploy the code and then install it onto your Confluence site by running these commands:

```
forge deploy
forge install
```

When prompted, select Confluence as the product and enter your site URL. You will be asked to approve the permissions listed in Section 1.7.

`forge install` is only needed once per site. After that, `forge deploy` alone is sufficient to push new code and the site picks up the changes automatically.

2.5 Environments

Forge provides three environments by default: development, staging and production. Each holds its own copy of the code and its own environment variables.

Command	What it does
<code>forge deploy</code>	Deploys to the default environment (development)
<code>forge deploy -e staging</code>	Deploys to staging
<code>forge deploy -e production</code>	Deploys to production
<code>forge environments list</code>	Shows environments and installed versions

Note: API Keys do not carry over across environments. Each one needs its own keys set, as described in Section 1.3.

Recommendation:

Use development for testing changes, staging for the stable version, and production for the version users can actually access (if you're solo developing this app).

If you're in a team, creating your own environment to deploy changes to is recommended:

1. In the root folder run `forge environments create` to create your own development environment. You will work in this environment. Think of it as something like your personal branch on git.

You will install what's in this environment onto your developer site. The combination of using your development environment and installing your version of the app on your site ensures that you don't overwrite or interfere with other people's code and you only install your version of the app to your developer site.

2. Run `forge deploy -e <your-environment-name>` to deploy to your development environment.

If you deploy to someone else's development environment you will overwrite the code in their development environment.

3. Run `forge install -e <your-environment-name>` to install the app (that's in your development environment) to your developer site.

Select confluence as the product and install ConfluenceGuessr onto your personal Atlassian developer site.

Once the app is installed on a site, the site picks up the new app changes you deploy without needing to rerun the `install` command.

You'll be able to see the app on the left navigation bar in Apps and Your apps.

It should say ConfluenceGuessr - your-environment-name (Dev).

Now you can start developing and working on ConfluenceGuessr.

Forge install overwrites everything on a given website so you can't share developer sites with your team as you'd be overwriting each other's app all the time.

2.6 Deploying frontend changes

Backend changes only require `forge deploy` to be run. Frontend changes require a rebuild first by running the following commands:

```
cd static
npm run build
cd ..
forge deploy
```

2.7 Verifying a deployment

Before deploying, two checks that are the most important can be run using these commands:

```
forge lint
npm test
```

`forge lint` validates the manifest and the app code. `npm test` runs the unit test suite.

After deploying, open the app from the Apps menu in the left navigation of the Confluence site and start a game. If questions generate and the count matches what was selected, the deployment successfully works.

Note: If the app loads, but the generation fails, check the API keys are set for that environment. As described in Section 1.4, the app falls back to non-AI questions rather than showing an error, so it is important to check for a missing key.

2.8 Troubleshooting

Syptom	Cause	Fix
<code>forge deploy</code> reports missing <code>index.html</code>	Frontend was never built	Run <code>npm run build</code> in <code>static</code>
Frontend changes not appearing	Build not run before deploy	Build, then deploy
App not visible in Confluence	Not installed on the site	Run <code>forge install</code>
Permission prompt appears mid-game	Scopes have changed since install	Run <code>forge install --upgrade</code>
Questions generate but quality is poor	API keys not set for that particular environment	Set keys as per section 1.3

3. Known limitations and future work

3.1 Question generation limits

The question generation logic has limits that keep it inside Forge's execution budget and the AI providers' free tier allowances. All are defined as constants inside `src/questionGen.mjs` and can be changed.

Limit	Value	Reason
Distinct pages user per game	10	Page content dominates token usage and without this cap, large page selection exceeds Groq's per minute limit
Page text sent to the AI	900 characters	Keeps the request small enough so that two batches run without hitting the limit

Pages per AI request	5	Brings down request size against the number of request made
Question counts offered	5, 10, 15, or 20	Defined in frontend and can be extended

This means that a player who selects 50 pages will still only have questions generated from 10 of them, chosen at random each game. Replaying produces a different set of questions.

Note: Raising the page or character limit increases token usage. On the Groq free tier this will cause rate limit errors as found during testing. Both values were chosen to fit inside the 8000 tokens per minute budget.

3.2 Pages that produce poor questions

Question quality is correlated to the pages selected by the user. The app requires readable text, and it rejects text that would make an unanswerable question.

Content that is filtered out is:

Filtered	Why
Headings and field labels	Too short to identify a page
Confluence errors	Not readable content
Raw markdown syntax	Appears when markdown is pasted into a page as plain text; does not help with the discovery of the page
Image filenames and URLs	Not readable
Text containing the page title	This reveals the answer

A clue must be between 40 to 300 characters and contain at least 8 words. A hint has a lower bar of 20 characters and 4 words.

Confluence template pages, which are mostly field labels with very little to no prose contribute few or no questions at all. Pages with written content produce noticeably better games.

Recommendation: Point players at spaces containing actual content like documentation, meeting notes or specifications rather than blank templates.

3.3 Limits of image question

Image questions are capped and filtered as well to stay inside Forge's response limit.

Limit	Value	Reason
% share of total questions	30%	Keeps the games mostly text based
Minimum image size	10 KB	Files below this size are icons and banners rather than actual content
Maximum image size	400 KB	Images are sent as base64 which adds approximately a third to the file size
Total image data per game	2 MB	Forge caps a frontend response at 5 MB

There are two further exclusions which apply. Images whose filename suggests that they are decorators, such as `header`, `banner` or `logo` are skipped because Confluence attaches default artwork to every page created from a template. Images of an identical file size appearing on more than one page are also skipped to mitigate the risk of these artworks appearing as well.

Note: If a page which contains no attached images will produce a text only game.

3.4 Generation time budget

Forge terminates a function after just 25 seconds. The app is built to work within that limit:

Budget	Value
Total generation	18 seconds
Image question probing and downloads	6 seconds
Frontend timeout before showing error	30 seconds

If the generation approaches 18 seconds, the app stops making AI requests and fills the remaining questions locally instead. The player still receives the full number of questions they requested.